NIYOJAN: An End-to-End Agentic Decision Intelligence Framework for Strategic Demand Planning

Rohit Khatri
Department of Computer Science & Engineering
MIT School of Computing, Loni Kalbhor
rohi7.forstudy@gmail.com

Siddharth Suryawanshi
Department of Computer Science & Engineering
MIT School of Computing, Loni Kalbhor
sidsurya1c11@gmail.com

Mahesh Swami
Department of Computer Science & Engineering
MIT School of Computing, Loni Kalbhor
swamimahesh256@gmail.com

Satyanarayan Mohapatro
Department of Computer Science & Engineering
MIT School of Computing, Loni Kalbhor
satyaamohapatro@gmail.com

Prof. Bhushan Bhoske (Mentor)
Department of Computer Science & Engineering
MIT School of Computing, Loni Kalbhor
bhushan.bhoske@mituniversity.edu.in

***Abstract*—Demand forecasting is widely used in supply chain planning, yet many systems provide only numerical predictions with limited decision support. This paper presents *NIYOJAN*, an agentic AI–driven decision intelligence system designed to transform demand forecasts into actionable planning insights. The proposed framework combines an LSTM-based forecasting model with deterministic inventory risk evaluation and an agentic reasoning layer capable of orchestrating structured analytical workflows. The system incorporates scenario simulation mechanisms to evaluate inventory outcomes under varying demand conditions, enabling planners to perform what-if analysis and risk assessment. A generative AI module provides human-readable explanations and strategic planning insights while maintaining separation from deterministic decision logic. The complete system is deployed using a cloud-based architecture with containerized services, demonstrating scalability and real-world applicability. Experimental results show reliable forecasting performance while significantly improving interpretability and planning capability compared to prediction-centric systems.**

***Index Terms*—Demand Forecasting, Agentic AI, Decision Intelligence, Strategic Planning, LSTM, Generative AI, Supply Chain Analytics, Cloud Deployment**

I. Introduction

Demand forecasting plays a foundational role in supply chain management, directly influencing inventory control, procurement, production scheduling, and customer service levels. Inaccurate forecasts often result in stockouts, excess inventory, and increased operational costs. Traditional statistical forecasting methods such as moving averages and ARIMA have been widely adopted due to their simplicity, yet they struggle to model nonlinear patterns and long-term temporal dependencies present in real-world sales data.

Recent advances in machine learning and deep learning, particularly recurrent neural networks such as Long Short-Term Memory (LSTM), have demonstrated improved forecasting accuracy by capturing complex temporal dynamics. Despite these advances, most forecasting systems remain prediction-centric, producing numerical outputs that are difficult for planners to interpret and act upon. In practice, decision-makers require not only accurate forecasts but also actionable recommendations, transparency, and trust in the system outputs.

Furthermore, many existing academic studies focus primarily on model-level performance and neglect system-level concerns such as explainability, deployment, and user interaction. The lack of integrated decision support often limits the adoption of advanced forecasting models in operational environments.

To address these limitations, this paper proposes *NIYOJAN*, an end-to-end decision intelligence system designed to transform demand forecasts into interpretable decisions. The key contributions of this work are summarized as follows:

- Design of an integrated decision intelligence architecture combining forecasting models with agentic AI reasoning workflows.
- Implementation of an LSTM-based demand forecasting model using a rolling prediction strategy.
- Development of a deterministic inventory risk evaluation mechanism for identifying stockout and overstock conditions.
- Integration of a scenario simulation engine that enables what-if analysis for inventory planning.
- Incorporation of an agentic reasoning layer that orchestrates analytical tools and generative AI modules to produce structured strategic insights.
- Deployment of the complete system using containerized infrastructure and cloud-based services.

II. Related Work

Demand forecasting has been extensively studied in the context of supply chain management. Douaioui *et al.* conducted a comprehensive review of machine learning and deep learning models for demand forecasting, highlighting the growing dominance of LSTM-based approaches over traditional statistical models [1]. Their analysis emphasizes improved accuracy but also notes challenges related to interpretability and practical deployment.

Pan *et al.* proposed an LSTM model augmented with an attention mechanism for retail demand forecasting, demonstrating superior performance compared to ARIMA and XGBoost baselines [2]. While effective at capturing temporal dependencies, the proposed approach remains focused on prediction accuracy and does not address downstream decision-making.

Jahin *et al.* introduced an explainable multi-channel data fusion network for supply chain demand forecasting, incorporating interpretability directly into the deep learning architecture [3]. Although model-level explainability is improved, the approach does not explicitly translate forecasts into business actions.

The potential role of large language models (LLMs) in supply chain management has been examined by Aghaei *et al.*, who highlight their usefulness in decision support, reporting, and scenario analysis rather than autonomous decision-making [4]. This perspective motivates the use of generative AI as an assistive component rather than a primary decision engine.

In contrast to existing work, the proposed NIYOJAN system focuses on system-level decision intelligence by combining forecasting accuracy, explainable decision logic, optional generative insights, and production-ready deployment.

III. System Architecture and Proposed Methodology

The NIYOJAN system follows a modular architecture composed of multiple interconnected layers responsible for forecasting, decision intelligence, analytical reasoning, and strategic planning. The architecture integrates predictive modeling, deterministic analytical computation, and agentic reasoning workflows to transform numerical forecasts into actionable planning insights.

A. Overall Architecture

The overall system architecture is illustrated in Fig. 1. The architecture consists of the user interface layer, backend API layer, forecasting engine, decision intelligence layer, and the agentic AI intelligence hub responsible for strategic planning.

Together, these tightly integrated modules form a continuous pipeline that elevates raw historical data into actionable business intelligence. While the foundational layers handle data ingestion, LSTM-based predictions, and deterministic risk classification, the agentic hub acts as the cognitive core. By dynamically orchestrating RAG-based knowledge retrieval, mathematical scenario simulations, and constrained LLM reasoning, the system successfully isolates quantitative execution from natural language generation, ensuring all executive-level procurement strategies remain mathematically sound and hallucination-free.

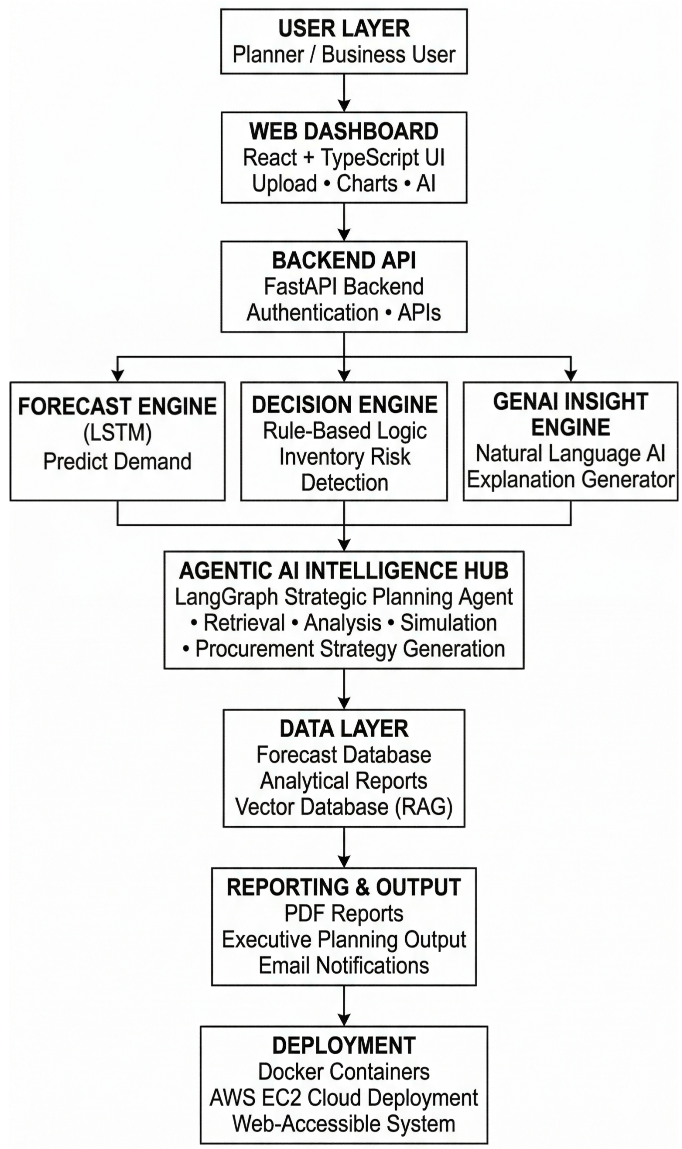

Fig. 1. System architecture of the proposed NIYOJAN framework integrating forecasting, decision intelligence, and agentic AI strategic planning.

B. Demand Forecasting Engine

The forecasting layer employs an LSTM network trained on historical sales data. Given an input sequence $\{x_{t-n}, \ldots, x_t\}$, the LSTM learns a function:

$$\hat{y}_{t+1} = f_{\text{LSTM}}(x_{t-n:t}) \tag{1}$$

where $\hat{y}_{t+1}$ represents the predicted demand for the next time step. A rolling forecast strategy is adopted, where predicted values are appended to the input sequence for multi-step forecasting.

C. Decision Logic Layer

Forecast outputs are transformed into actionable decisions using a rule-based engine. A risk ratio is computed as:

$$R = \frac{\hat{y}_{t+1}}{S_t} \tag{2}$$

where S_t denotes current stock level. Threshold-based rules classify inventory risk into *High*, *Medium*, or *Low*, corresponding to actions such as restocking, holding, or reducing inventory.

D. Agentic Reasoning Layer

NIYOJAN enhances decision intelligence through an agentic reasoning layer that orchestrates multi-step analytical workflows. This layer coordinates data retrieval, scenario simulation, and strategic planning while explicitly separating deterministic math from LLM reasoning. Consequently, the system processes structured context—such as demand trends and inventory gaps—to deliver reliable, flexible planning insights without computational hallucination.

E. Scenario Simulation Engine

The system incorporates a deterministic scenario simulation engine designed to evaluate inventory outcomes under varying demand conditions. This module enables planners to perform counterfactual analysis by modifying forecast values and analyzing their impact on inventory risk.

Demand variation scenarios are computed as:

$$NewDemand = Forecast \times (1 + \frac{p}{100}) \tag{3}$$

where p represents the percentage change applied to the baseline demand forecast.

Inventory planning metrics are then calculated using classical supply chain models:

$$SafetyStock = Z \times \sigma \times \sqrt{LeadTime} \tag{4}$$

$$ROP = (Demand \times LeadTime) + SafetyStock \tag{5}$$

$$Gap = ROP - CurrentStock \tag{6}$$

These deterministic computations allow the system to evaluate inventory risks under simulated demand scenarios, enabling planners to assess potential stock shortages or excess inventory.

F. Agentic Workflow

The agentic workflow organizes analytical reasoning into a structured decision pipeline. User queries or forecasting outputs trigger the reasoning process, which begins with intent detection and contextual analysis. The system then routes tasks to specialized analytical modules responsible for performing retrieval, statistical analysis, scenario simulation, or strategic planning.

The overall reasoning pipeline can be summarized as follows:

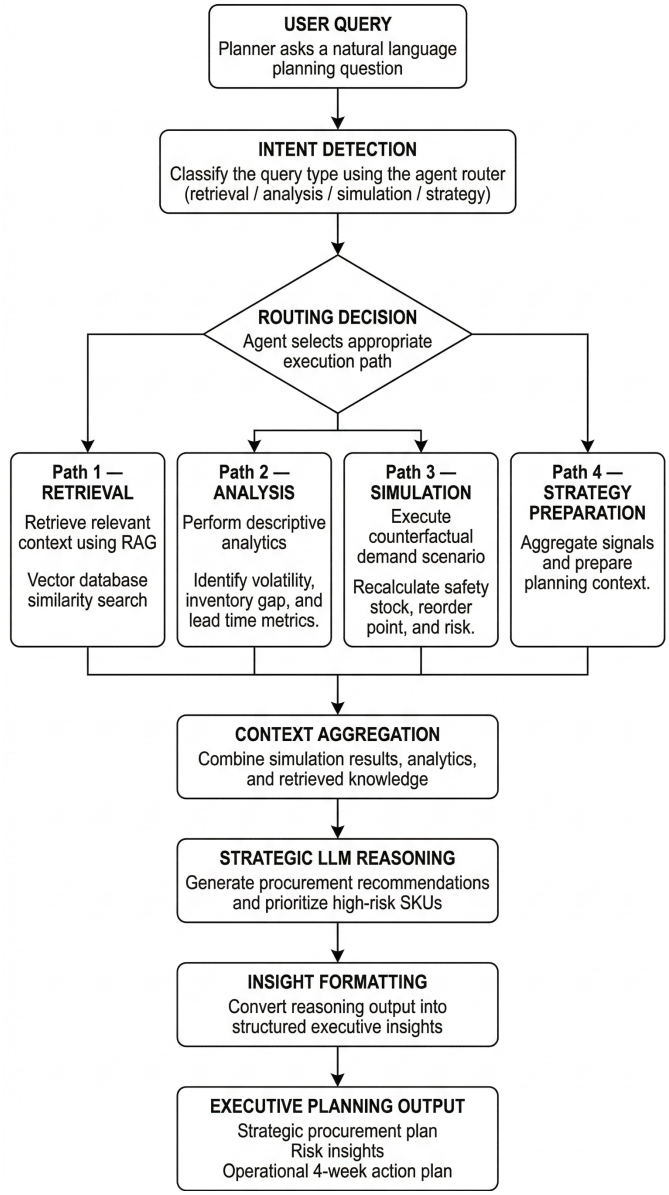

Fig. 2. Agentic AI workflow for strategic demand planning. The workflow includes intent classification, conditional routing, analytical tools, scenario simulation, and LLM-assisted strategic reasoning.

User Query or Forecast Output $\rightarrow$ Context Analysis $\rightarrow$ Analytical Tool Execution $\rightarrow$ Scenario Simulation $\rightarrow$ Strategic Planning $\rightarrow$ Insight Generation.

This structured orchestration ensures that numerical computations remain deterministic while generative AI components focus on explanation and strategic interpretation.

G. Generative AI Insight Engine

An optional generative AI module is integrated to provide natural language explanations of forecasts and decisions. The module operates on structured inputs and generates descriptive summaries without influencing deterministic decision outcomes.

H. Deployment Workflow

The entire system is containerized using Docker, with separate containers for frontend, backend, and auxiliary services. Deployment is carried out on AWS EC2, enabling scalable and reproducible execution.

IV. Dataset Description and Preprocessing

The dataset used in this study consists of historical retail sales records collected at a weekly granularity. Each record includes a product identifier, time index (week), and corresponding sales quantity, enabling time-series demand forecasting for individual products.

The dataset is derived from a retail sales dataset commonly used in demand forecasting research. Prior to model training, missing values were handled appropriately, and extreme outliers were removed to improve data quality.

The dataset contains approximately 50,000 weekly sales records across 120 products collected over a three-year period.

Sales quantities were normalized using Min-Max scaling to ensure consistent input ranges for the neural network. The dataset was split chronologically into training (70%), validation (10%), and testing (20%) sets to preserve temporal order and prevent information leakage during forecasting experiments.

V. Model Implementation

The forecasting component of NIYOJAN is implemented using a Long Short-Term Memory (LSTM) network due to its ability to capture temporal dependencies in sequential data. The model consists of two stacked LSTM layers with 64 hidden units followed by a dense output layer for demand prediction.

Input sequences are generated using sliding windows over historical sales data. The model is trained using the Adam optimizer with a learning rate of 0.001 and Mean Squared Error (MSE) as the loss function. TensorFlow and Keras frameworks are used for implementation. Multi-step forecasts are generated using a rolling prediction strategy in which predicted values are iteratively appended to the input sequence.

VI. Experimental Setup and Evaluation Metrics

The experimental evaluation of the proposed NIYOJAN framework was conducted in a cloud-based computing environment. The experiments were executed on an Amazon Web Services (AWS) EC2 instance equipped with 8 virtual CPUs and 32 GB of system memory. The forecasting models were implemented using Python with TensorFlow and Keras libraries.

The dataset was divided chronologically into training, validation, and testing subsets to ensure realistic forecasting evaluation and prevent data leakage. Approximately 70% of the dataset was used for training, 10% for validation, and the remaining 20% for testing.

To assess forecasting accuracy, two widely used time-series evaluation metrics were employed:

Mean Absolute Error (MAE):

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{7}$$

MAE measures the average absolute difference between predicted and actual demand values, providing an interpretable estimate of prediction deviation.

Root Mean Squared Error (RMSE):

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{8}$$

RMSE penalizes larger errors more heavily and therefore highlights instances of high forecasting deviation.

In addition to forecasting accuracy, qualitative evaluation was performed to assess the effectiveness of the decision intelligence layer and the agentic reasoning workflow in supporting strategic planning tasks.

VII. Results and Analysis

This section evaluates the performance of the proposed NIYOJAN system, focusing on forecasting accuracy, decision-layer effectiveness, and system-level usability. Table **??** summarizes the forecasting performance of the LSTM model.

A. Forecasting Performance

The forecasting accuracy of NIYOJAN was assessed using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), which are standard metrics for time-series demand prediction. Evaluation was performed on weekly, product-level sales data.

TABLE I
Comparison of Forecasting Models

Model	MAE	RMSE
Moving Average	12.8	16.5
ARIMA	14.2	19.1
LSTM (NIYOJAN)	9.8	12.4

Table I compares the forecasting performance of the proposed LSTM-based NIYOJAN model with traditional statistical approaches such as Moving Average and ARIMA. The results demonstrate that the LSTM model achieves lower MAE and RMSE values, indicating improved predictive accuracy and better capability to capture temporal demand patterns in retail sales data.

Baseline models including Moving Average and ARIMA were implemented using the same dataset and evaluation protocol to provide a fair comparison with the proposed LSTM-based forecasting model.

B. Decision Layer Effectiveness

NIYOJAN extends forecasting into decision intelligence by mapping predicted demand to *High*, *Medium*, or *Low* inventory risk categories using a rule-based decision engine. This abstraction enables rapid identification of stockout and overstock scenarios, improving interpretability and operational clarity over forecast-only outputs.

C. Impact of Generative AI Insights

The optional generative AI module provides concise natural language explanations aligned with forecast trends and decision outcomes, reducing cognitive effort for users. The module operates strictly as an assistive layer and does not influence deterministic decision logic.

D. System-Level Observations

Overall, NIYOJAN maintains competitive forecasting accuracy while significantly enhancing decision support and explainability, resulting in a system better suited to real-world demand planning than traditional prediction-centric approaches.

E. Strategic Planning Capability

Beyond forecasting accuracy, the system enables planners to analyze inventory conditions through structured analytical workflows. By combining deterministic simulations with contextual reasoning, the architecture allows decision-makers to evaluate demand fluctuations, identify high-risk inventory items, and develop proactive planning strategies.

The integration of forecasting models with analytical reasoning modules significantly improves the interpretability and operational usability of the system compared to prediction-centric approaches.

VIII. Deployment and Scalability

The NIYOJAN system is deployed using a containerized architecture based on Docker. The application components, including the frontend interface, backend API, and analytical modules, are packaged as independent services.

Deployment is performed on an AWS EC2 instance, enabling scalable cloud-based execution. The backend services are implemented using FastAPI and expose forecasting and analytical functionalities through REST APIs. This architecture ensures portability, reproducibility, and efficient system scalability.

IX. Discussion

The experimental results demonstrate that integrating predictive modeling with decision intelligence mechanisms can significantly enhance the practical usefulness of demand forecasting systems. While forecasting accuracy remains an important component, operational decision-making requires additional contextual interpretation of predicted demand patterns.

The proposed NIYOJAN framework addresses this challenge by combining three complementary capabilities: predictive forecasting, deterministic analytical reasoning, and agentic AI-based strategic planning. The forecasting module provides quantitative demand estimates, while the rule-based decision layer translates these predictions into interpretable inventory risk categories.

The introduction of the agentic reasoning layer further extends the system's capabilities by enabling structured analytical workflows, including scenario simulation and contextual planning analysis. By separating deterministic mathematical computation from language model reasoning, the architecture minimizes the risk of unreliable or hallucinated outputs while still benefiting from the interpretive capabilities of generative AI.

From a practical perspective, this hybrid architecture allows planners to not only anticipate demand fluctuations but also explore potential response strategies under different demand conditions. Consequently, the system moves beyond predictive analytics toward actionable decision intelligence for supply chain management.

X. Conclusion and Future Work

This paper presented NIYOJAN, an agentic AI-driven decision intelligence system designed to enhance strategic demand planning in supply chain environments. The proposed architecture integrates LSTM-based demand forecasting, deterministic inventory risk evaluation, scenario simulation, and agentic reasoning workflows to transform numerical predictions into actionable planning insights.

The experimental evaluation demonstrates that the forecasting module achieves reliable prediction performance while the decision intelligence layer improves interpretability and operational usability. Furthermore, the agentic AI intelligence hub enables planners to perform scenario analysis and strategic planning tasks by orchestrating multiple analytical tools within a structured workflow.

The results indicate that combining predictive analytics with deterministic reasoning and AI-assisted planning can significantly improve the practical effectiveness of demand forecasting systems in real-world supply chain applications.

Future research will focus on extending the system toward multi-agent collaborative planning environments, real-time data streaming pipelines, and reinforcement learning–based procurement optimization strategies. Additionally, integrating adaptive decision thresholds and dynamic risk modeling could further enhance the robustness of the proposed framework.

References

[1] K. Douaioui, R. Oucheikh, O. Benmoussa, and C. Mabrouki, "Machine learning and deep learning models for demand forecasting in supply chain management: A critical review," *Applied System Innovation*, vol. 7, no. 5, p. 93, 2024.

[2] G. Pan, B. Xue, Y. Lu, and X. Li, "A deep learning-based demand forecasting method for retail supply chains using LSTM and attention mechanism," in *Proceedings of the International Conference on Management Science and Engineering Management*, 2025.

[3] J. Jahin et al., "MCDFN: Supply chain demand forecasting via an explainable multi-channel data fusion network," *arXiv preprint arXiv:2405.15598*, 2024.

[4] A. Aghaei et al., "The potential of large language models in supply chain management," *arXiv preprint arXiv:2501.15411*, 2025.

[5] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M4 Competition: Results, findings, conclusion and way forward," *International Journal of Forecasting*, vol. 34, no. 4, pp. 802–808, 2018.

[6] R. Bandara, C. Bergmeir, and S. Smyl, "Forecasting across time series databases using recurrent neural networks on groups of similar series," *Expert Systems with Applications*, vol. 140, 2020.

[7] A. Fildes, N. Ma, and S. Kolassa, "Retail forecasting: Research and practice," *International Journal of Forecasting*, vol. 35, no. 1, pp. 1–9, 2019.

[8] J. Benesty, J. Chen, Y. Huang, and I. Cohen, "Inventory management and demand forecasting using machine learning techniques," *IEEE Access*, vol. 9, pp. 112234–112245, 2021.